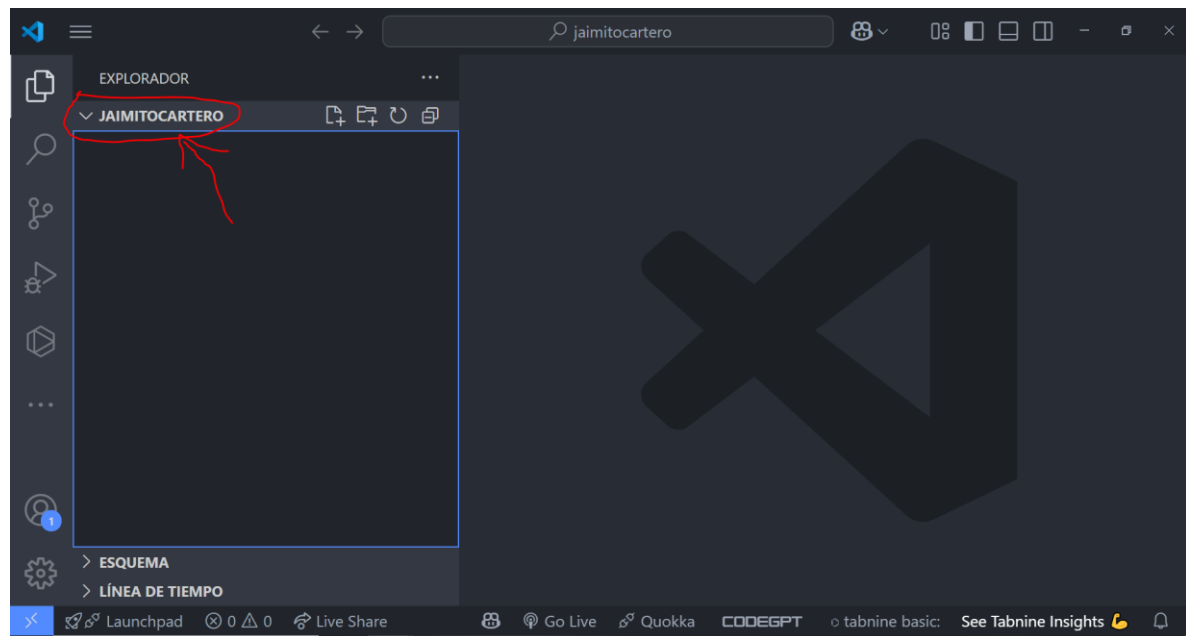


Crear proyecto envío de correo jaimitocartero

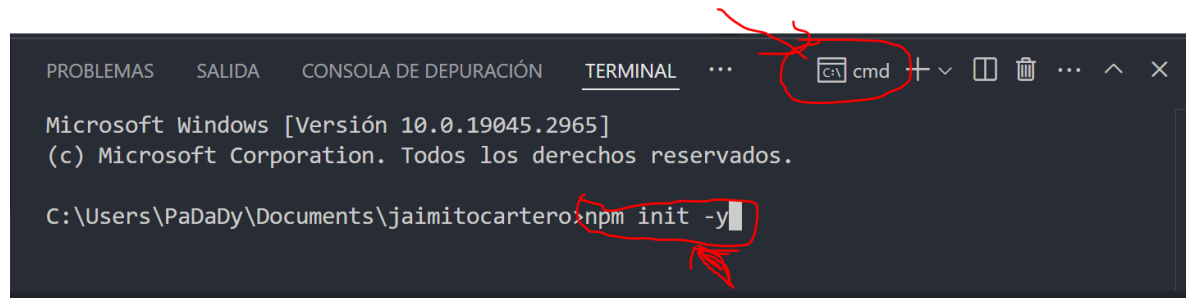
1 se crear carpeta raíz en documento



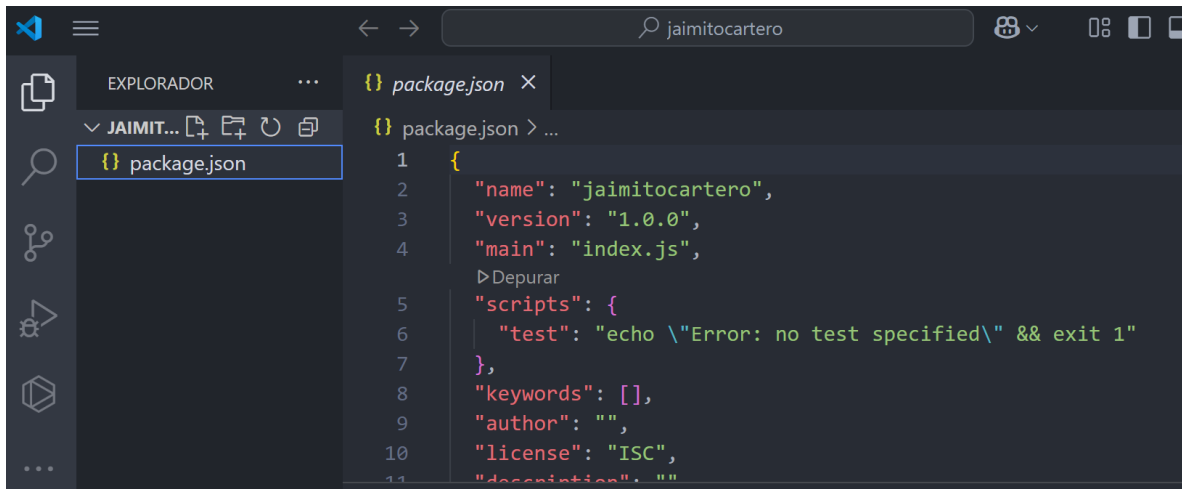
2 visual code con la carpeta raíz se integran



3 se instala node dentro de la carpeta raíz por medio de la terminal (cmd) promand comand, con el código **npm init -y**

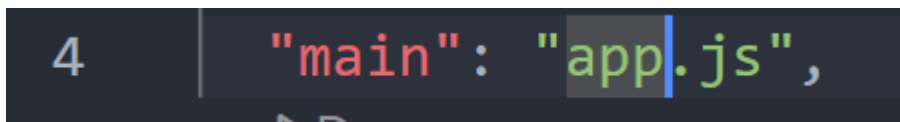


Al instalar node se vera algo como así



```
1 {  
2   "name": "jaimitocartero",  
3   "version": "1.0.0",  
4   "main": "index.js",  
5   "scripts": {  
6     "test": "echo \"Error: no test specified\" && exit 1"  
7   },  
8   "keywords": [],  
9   "author": "",  
10  "license": "ISC",  
11  "description": ""
```

3 se configura el package.json línea 4 mi archivo principal se llamara app.js



```
4   "main": "app.js",
```

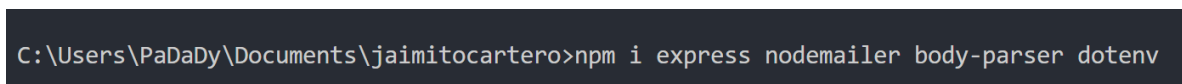
Se configura la línea 6 en los primeros parenticis quedara “dev” y los segundos “node --watch app.js”



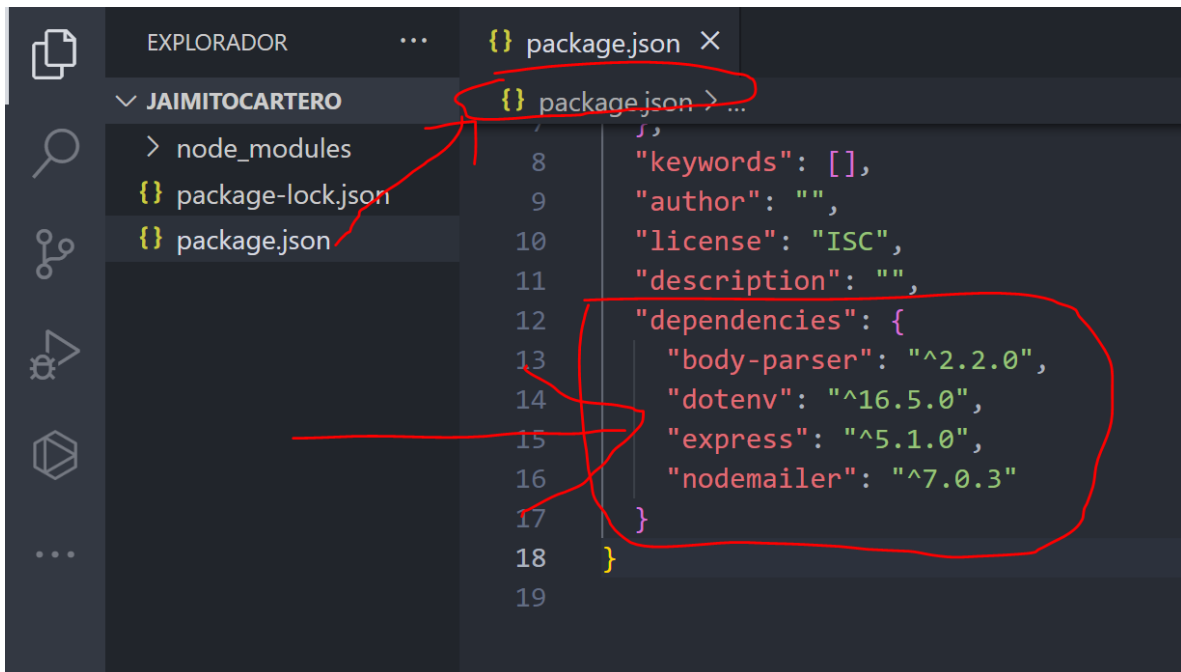
```
5   "scripts": {  
6     "dev": "node --watch app.js"
```

4 Instalación de librerías necesarias

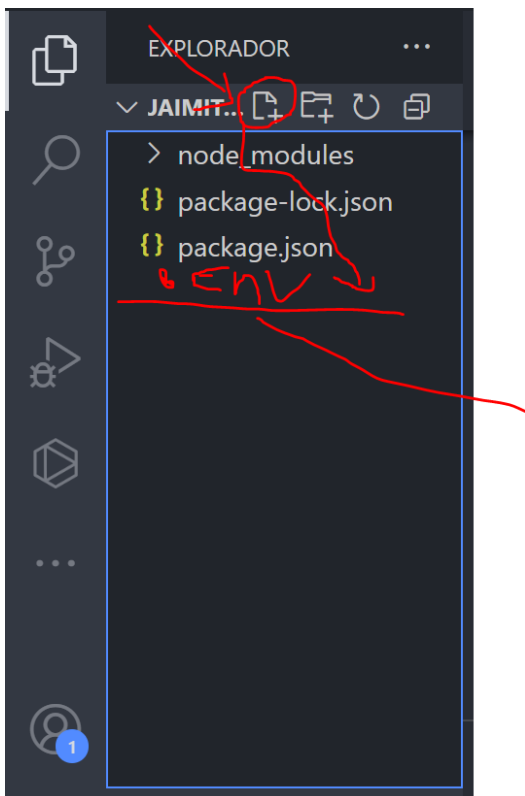
npm i express nodemailer body-parser dotenv



```
C:\Users\PaDaDy\Documents\jaimitocartero>npm i express nodemailer body-parser dotenv
```



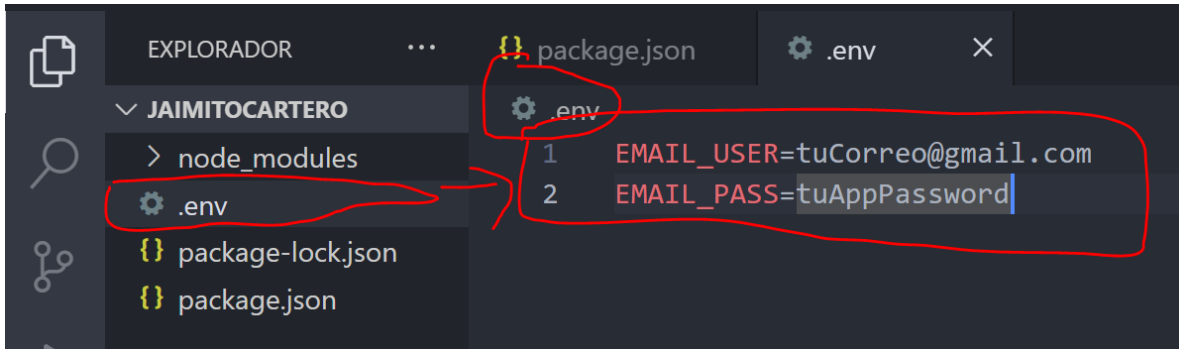
5 paso crear un archivo .env en la cual este contendrá todos los datos sensibles, para esto instalamos la librería dotenv, para el manejo de este archivo.



En el vamos a escribir los datos

[EMAIL_USER=tuCorreo@gmail.com](#) aquí si espacio el correo personal

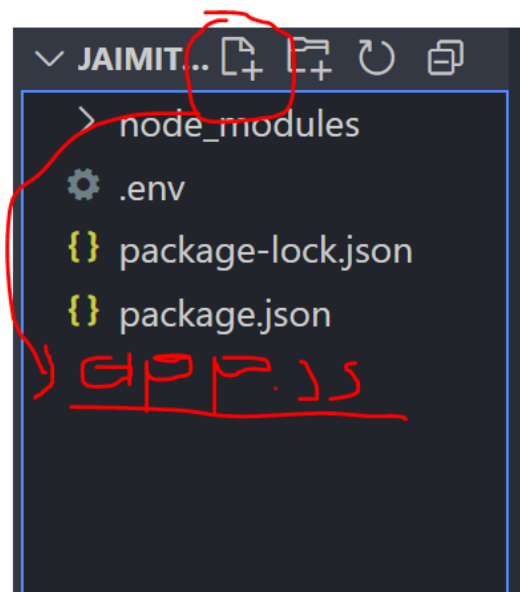
EMAIL_PASS=tuAppPassword aquí la contraseña de ese correo



Ahora el paso 6 es crear el archivo principal que lo llamamos app.js

Este archivo va contener el servidor para que pueda correr el proyecto y las rutas necesarias para poder acceder desde la web

6. crear app.js



Creamos un nuevo archivo en la carpeta raíz que lo llamaremos app.js

Dentro de el vamos a escribir todo estos códigos

```
const express = require('express');

const bodyParser = require('body-parser');

const nodemailer = require('nodemailer');
require('dotenv').config();

const app = express();

const PORT = 3000; // para el puerto


// Middleware
app.use(bodyParser.urlencoded({ extended: false })); //
app.use(bodyParser.json()); // para pasar de JSON a HTML
app.use(express.static('public')); // para crear una carpeta estatica public


// Ruta para enviar correo
app.post('/send-email', async (req, res) => {
  const { nombre, email, mensaje } = req.body;


  // Configurar transporte
  const transporter = nodemailer.createTransport({
    service: 'gmail',
    auth: {
      user: process.env.EMAIL_USER,
      pass: process.env.EMAIL_PASS
    }
  });

});


// Opciones del correo
```

```
const mailOptions = {  
  from: process.env.EMAIL_USER,  
  to: email,  
  subject: 'Correo desde Node.js',  
  text: `Hola ${nombre},\n\nGracias por tu mensaje:\n${mensaje}`,  
  html: `

## Hola ${nombre}</h2><p>Gracias por tu mensaje:</p><p>${mensaje}</p>` }; try { await transporter.sendMail(mailOptions); res.json({ success: true, message: 'Correo enviado correctamente' }); } catch (error) { console.error(error); res.status(500).json({ success: false, message: 'Error al enviar el correo' }); } }); app.listen(PORT, () => { console.log(`Servidor corriendo en http://localhost:${PORT}`); });


```

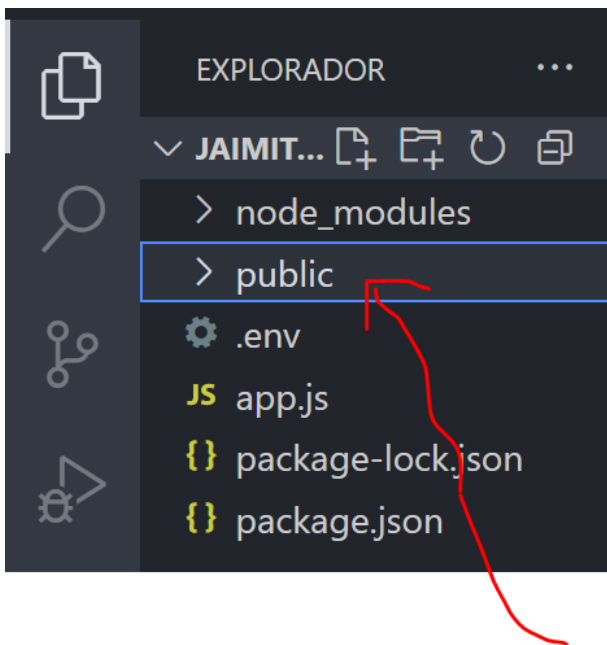
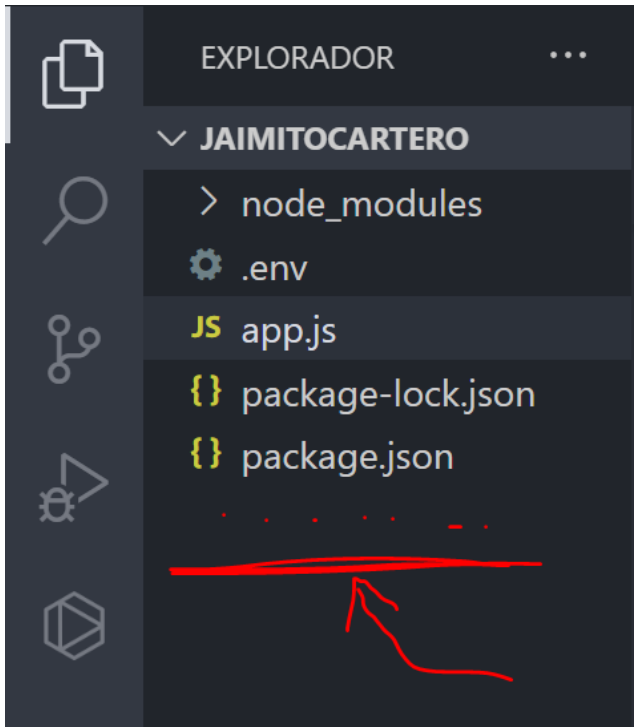
```
{ } package.json  .env  JS app.js  X
JS app.js > app.post('/send-email') callback
1  const express = require('express');
2  const bodyParser = require('body-parser');
3  const nodemailer = require('nodemailer'); 207.5k (gzipped: 56.4k)
4  require('dotenv').config(); 6.4k (gzipped: 2.8k)
5  const app = express();
6  const PORT = 3000; // configurar el puerto en 3000
7
8  // Middleware
9  app.use(bodyParser.urlencoded({ extended: false }));
10 app.use(bodyParser.json()); // para convertir json a html
11 app.use(express.static('public')); // crear una carpeta estatica
12
```

```
{ } package.json  .env  JS app.js  X
JS app.js > app.post('/send-email') callback
12
13 // Ruta para enviar correo
Tabnine | Edit | Test | Explain | Document
14 app.post('/send-email', async (req, res) => {
15     const { nombre, email, mensaje } = req.body;
16
17     // Configurar el correo con el .env
18     const transporter = nodemailer.createTransport({
19         service: 'gmail',
20         auth: {
21             user: process.env.EMAIL_USER,
22             pass: process.env.EMAIL_PASS
23         }
24     });
25
```

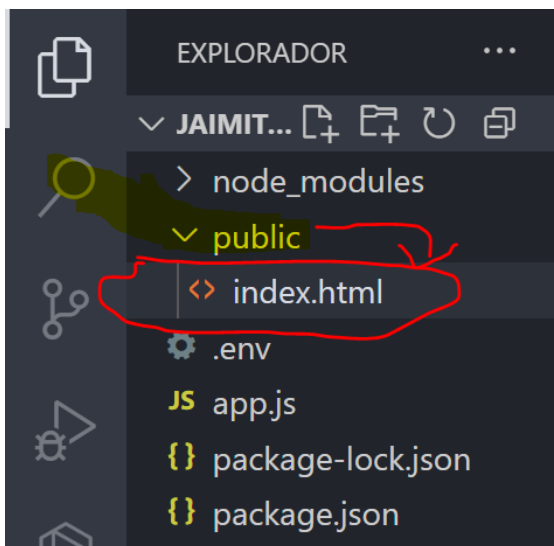
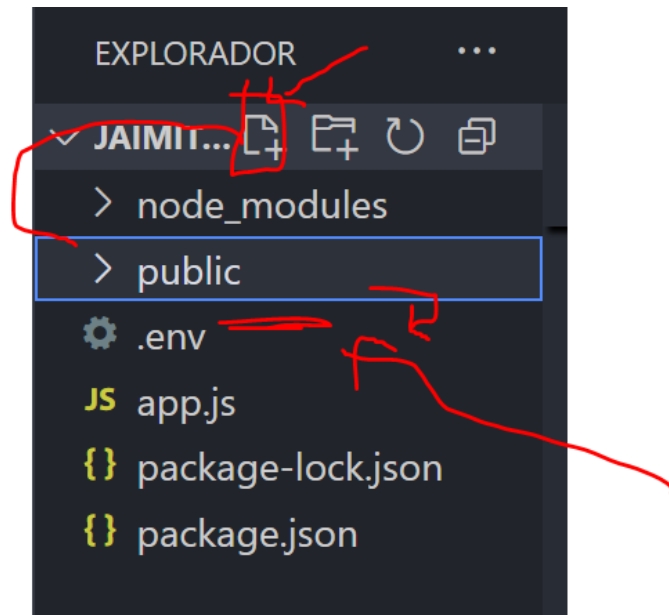
```
{ } package.json  .env  JS app.js  X
JS app.js > app.post('/send-email') callback
14  app.post('/send-email', async (req, res) => {
25
26      // configurar que dira el correo |
27      const mailOptions = {
28          from: process.env.EMAIL_USER,
29          to: email,
30          subject: 'Correo desde Node.js',
31          text: `Hola ${nombre},\n\nGracias por tu mensaje:\n${mensaje}`,
32          html: `<h2>Hola ${nombre}</h2><p>Gracias por tu mensaje:</p><p>${mensaje}</p>`
33      };
34
35      try {
36          await transporter.sendMail(mailOptions);
37          res.json({ success: true, message: 'Correo enviado
```

```
{ } package.json  .env  JS app.js  X
JS app.js > app.post('/send-email') callback
14  app.post('/send-email', async (req, res) => {
38      } catch (error) {
39          console.error(error);
40          res.status(500).json({ success: false, message: 'Error al
    enviar el correo' });
41      }
42  });
43  // este crear el servidor
    Tabnine | Edit | Test | Explain | Document
44  app.listen(PORT, () => {
45      console.log(`Servidor corriendo en http://localhost:${PORT}`);
46  });
47
```


7 vamos a crear una carpeta dentro de la carpeta raíz llamada **public**



8 vamos a crear un archivo llamado index.html



Este es un archivo estatico pero es el que va a poder visualizar el usuario final.

En este archivo vamos a escribir el siguiente código

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
  <title>Enviar Correo con Node.js</title>
</head>
<body>
  <h2>Formulario de Contacto</h2>
  <form id="contactForm">
    <label>Nombre:</label><br>
    <input type="text" name="nombre" required><br><br>

    <label>Email destino:</label><br>
    <input type="email" name="email" required><br><br>

    <label>Mensaje:</label><br>
    <textarea name="mensaje" required></textarea><br><br>

    <button type="submit">Enviar</button>
  </form>

  <p id="responseMsg"></p>
```

```
<script>
  const form = document.getElementById('contactForm');
  const responseMsg = document.getElementById('responseMsg');

  form.addEventListener('submit', async (e) => {
    e.preventDefault();
    const formData = new FormData(form);
    const data = Object.fromEntries(formData.entries());

    const res = await fetch('/send-email', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(data)
    });

    const result = await res.json();
    responseMsg.textContent = result.message;
  });
</script>
</body>
</html>
```

```
{ } package.json  .env  JS app.js  <> index.html X
public > <> index.html > html > head
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width,
6          initial-scale=1.0"/>
7      <title>Enviar Correo con Node.js</title>
8  </head>
9  <body>
10     <h2>Formulario de Contacto</h2>
11     <form id="contactForm">
12         <label>Nombre:</label><br>
13         <input type="text" name="nombre" required><br><br>
14         <label>Email destino:</label><br>
15         <input type="email" name="email" required><br><br>
16         <label>Mensaje:</label><br>
17         <textarea name="mensaje" required></textarea><br><br>
18         <button type="submit">Enviar</button>
19     </form>
20
21
22
```

```
23 <p id="responseMsg"></p>
24
25 <script>
26   const form = document.getElementById('contactForm');
27   const responseMsg = document.getElementById('responseMsg');
28
29   form.addEventListener('submit', async (e) => {
30     e.preventDefault();
31     const formData = new FormData(form);
32     const data = Object.fromEntries(formData.entries());
33
34     const res = await fetch('/send-email', {
35       method: 'POST',
36       headers: { 'Content-Type': 'application/json' },
37       body: JSON.stringify(data)
38     });
```

```
39
40     const result = await res.json();
41     responseMsg.textContent = result.message;
42   });
43 </script>
44 </body>
45 </html>
46
```

Pasos para configurar la cuenta si es Google ya que no deja con solo colocar la simple contraseña.

Activa la **verificación en dos pasos** de tu cuenta Google:

👉 <https://myaccount.google.com/security>

☐ Ve a **Contraseñas de aplicaciones**:

👉 <https://myaccount.google.com/apppasswords>

(puede pedirte que inicies sesión)

☐ Elige:

- App: "Correo"
- Dispositivo: "Otro" → ponle nombre, por ejemplo: NodeMailer
- Haz clic en "Generar"

☐ Google te dará una contraseña especial de 16 caracteres (como: abcd efgh ijkl mnop).